

Complementi di Programmazione

# Python: Funzioni

CdL Informatica - Università degli studi di Modena e Reggio Emilia  
AA 2024/2025

Angelo Ferrando

# Funzioni - definizione

In Python esistono le funzioni. A differenza dei metodi esse non sono innestate in una classe.

Si definiscono con la parola chiave **def**

La funzione è un blocco di codice (quindi indentato)

```
def funzione(arg1, arg2):
```

```
    print(arg1)
```

```
    print(arg2)
```

```
    return 1
```

# Funzioni - definizione

In Python esistono le funzioni. A differenza dei metodi esse non sono innestate in una classe.

Si definiscono con la parola chiave **def**

La funzione è un blocco di codice (quindi indentato)

**def funzione(arg1, arg2):**

**print(arg1)**

**print(arg2)**

**return 1**

Non si specificano i tipi degli argomenti  
e dei valori ritornati dalla funzione!

# Funzioni - esempio

```
def fib(n):
```

```
    a, b = 0, 1
```

```
    print(a)
```

```
    for i in range(n):
```

```
        print(b)
```

```
        a, b = b, a + b
```

# Funzioni - Argomenti opzionali

E' possibile definire funzioni con parametri opzionali che assumono un valore di default se non passati durante l'invocazione.

```
def funzione(arg1, arg2=0):
```

```
    print(arg1)
```

```
    print(arg2)
```

```
    return 1
```

Si specifica il valore di default usando =  
I parametri opzionali devono essere gli ultimi.

# Funzioni - Passaggio di argomenti

Normalmente si usa la notazione posizionale (positional arguments)

```
a = funzione(0,1)
```

```
def funzione(arg1, arg2=0):
```

```
    print(arg1)
```

```
    print(arg2)
```

```
    return 1
```

# Funzioni - Passaggio di argomenti

E' possibile anche specificare a quale parametro ci si riferisce (keyword arguments)

```
a = funzione(arg2=3,arg1=1)
```

```
def funzione(arg1, arg2=0):
```

```
    print(arg1)
```

```
    print(arg2)
```

```
    return 1
```

# Funzioni - Passaggio di argomenti

E' possibile mischiare entrambe le tecniche.

I positional argument vanno prima dei keyword argument in questo caso!

```
a = funzione(3,arg2=1)
```

```
def funzione(arg1, arg2=0):
```

```
    print(arg1)
```

```
    print(arg2)
```

```
    return 1
```

# Funzioni - Passaggio di argomenti

E' possibile 'spacchettare' sequenze (es. liste) e passare ogni elemento come positional argument.

Si usa l'**operatore \*** anteposto al dizionario

**a = [1,2,3,4,5]**

**funzione(\*a)** equivale a **funzione(1,2,3,4,5)**

Il numero di parametri deve essere lo stesso di quelli definiti nella funzione (al netto di quelli opzionali)

# Funzioni - Passaggio di argomenti

E' possibile 'spacchettare' dizionari e passare ogni elemento come keyword argument.

Si usa l'**operatore \*\* anteposto alla dizionario**

**a = {a:1, b:2, c:3}**

**funzione(\*\*a)** equivale a **funzione(a=1,b=2,c=3)**

# Funzioni - Parametri formali con \* e \*\*

Gli operatori \* e \*\* possono anche essere utilizzati nella definizione della funzione.

```
def funzione(*args, **kwargs):
```

```
    ...
```

# Funzioni - Parametri formali con \* e \*\*

Gli operatori \* e \*\* possono anche essere utilizzati nella definizione della funzione.

```
def funzione(*args, **kwargs):
```

```
    ...
```

In questo caso tutti i parametri posizionali passati alla funzione vengono condensati nella tupla **args**.

Tutti i keyword arguments vengono condensati in un dizionario **kwargs**

# Funzioni - Parametri formali con \* e \*\*

Gli operatori \* e \*\* possono anche essere utilizzati nella definizione della funzione.

```
def funzione(*args, **kwargs):
```

```
    ...
```

Utile se non si sa a priori il numero di parametri necessari.

# Funzioni - Parametri formali con \* e \*\*

Gli operatori \* e \*\* possono anche essere utilizzati nella definizione della funzione. **La funzione può comunque avere parametri espliciti**

```
def funzione(arg1, arg2, *args, **kwargs):
```

```
    ...
```

arg1 e arg2 non saranno presenti in args e kwargs

# Funzioni - Ricorsione

Come in altri linguaggi, in Python è possibile scrivere una funzione che chiama se stessa. **Ricorsione**

# Funzioni - Ricorsione

Come in altri linguaggi, in Python è possibile scrivere una funzione che chiama se stessa. **Ricorsione**

Remainder:

- CONDIZIONE DI STOP!

# Funzioni - Ricorsione

```
def fib(n):
```

```
    a, b = 0, 1
```

```
    print(a)
```

```
    for i in range(n):
```

```
        print(b)
```

```
        a, b = b, a + b
```



```
def fib(n):
```

```
    if (n == 1):
```

```
        print(1)
```

```
        return 0, 1
```

```
    a, b = fib(n-1)
```

```
    print(a+b)
```

```
    return b, a+b
```

# Funzioni - Ricorsione - Appiattare le liste

Provare usando la ricorsione a scrivere una funzione che appiattisca una lista di liste.

Aiutarsi con la funzione **hasattr**. Cercare nella documentazione

# Funzioni - Ricorsione - Appiattare le liste

Provare usando la ricorsione a scrivere una funzione che appiattisca una lista di liste.

```
def flatten(l):  
    res = []  
    if not hasattr(l, '__iter__'):  
        return [l]  
    for o in l:  
        res.extend(flatten(o))  
    return res
```

# Programmazione di ordine superiore

Si definisce un linguaggio come **di ordine superiore** se consente di utilizzare funzioni come valori.

Le funzioni possono essere passate ad altre funzioni

Possono essere ritornate come risultato di altre funzioni

# Programmazione di ordine superiore

```
def get_print_help_function(lang):
```

```
    def eng_help():
```

```
        print('help')
```

```
    def ita_help():
```

```
        print('aiuto')
```

```
    if lang == 'eng':
```

```
        return eng_help
```

```
    else:
```

```
        return ita_help
```

Il valore di ritorno è una funzione.  
Notare l'assenza di ()

# Programmazione di ordine superiore

```
def print_function_result(func):  
    print(f'il risultato è {func()}')
```

La funzione prende in ingresso una funzione che chiamerà al suo interno.

# Programmazione di ordine superiore - CLOSURE

Supponiamo di voler avere una funzione inizializzabile che poi stampi sempre lo stesso messaggio quando chiamata.

Si potrebbe creare una funzione che preso il parametro di inizializzazione restituisce una funzione che stampa sempre quel messaggio.

# Programmazione di ordine superiore - CLOSURE

Si potrebbe creare una funzione che preso il parametro di inizializzazione restituisce una funzione che stampa sempre quel messaggio.

```
def print_msg(msg):  
    def printer():  
        print(msg)  
    return printer
```

# Programmazione di ordine superiore - CLOSURE

Si potrebbe creare una funzione che preso il parametro di inizializzazione restituisce una funzione che stampa sempre quel messaggio.

```
def print_msg(msg):
```

```
    def printer():
```

```
        print(msg)
```

```
    return printer
```

Printer deve accedere a **msg** che non fa parte della funzione stessa.

In Python è possibile

# Programmazione di ordine superiore - CLOSURE

Si potrebbe creare una funzione che preso il parametro di inizializzazione restituisce una funzione che stampa sempre quel messaggio.

```
def print_msg(msg):
```

```
    def printer():
```

```
        print(msg)
```

```
    return printer
```

Concetto di **CLOSURE**:

Le funzioni innestate possono accedere alle variabili delle funzioni madri.

Tali variabili vengono incapsulate nelle funzioni figlie in modo che possano essere utilizzate anche quando la funzione madre termina.

# Programmazione di ordine superiore - Lambda

Partiamo da una funzione di Python molto esplicativa del concetto di ordine superiore.

La funzione **map()**

Essa prende in ingresso una funzione e una sequenza di oggetti.

Applica tale funzione a ogni oggetto e ne ritorna il risultato (come lista)

# Programmazione di ordine superiore - Lambda

In certi casi la funzione da applicare è molto semplice, per esempio il quadrato.

```
def quad(a)  
    return a**2
```

```
map(quad, [1,2,3])
```

# Programmazione di ordine superiore - Lambda

Python permette di passare la funzione senza definirla in un altro punto del codice.

Viene detta lambda function (o anche funzione anonima)

Sintassi: **lambda** argomenti: valore di ritorno

Può contenere un'unica istruzione che sarà il valore di ritorno (non si mette return)

```
map(lambda x:x**2, [1,2,3])
```

# Generatori

Abbiamo visto che in Python si può iterare su qualsiasi oggetto che abbia un metodo `__iter__` che ritorni un iteratore.

Questo implica la costruzione di una classe con tale metodo

# Generatori

I **Generatori** semplificano questo lavoro.

Possono essere sia funzioni sia espressioni

In entrambi i casi possono essere utilizzate in un ciclo

# Generatori - Funzioni generatrici

Le funzioni Generatrici generano ad ogni iterazione l'elemento successivo.

E come se ritornassero una serie di valori invece che un unico risultato.

Si creano attraverso il comando **yield seguito dal valore da tornare**

# Generatori - Funzioni generatrici

Quando una funzione contiene il comando `yield`, essa viene trattata da Python come un Generatore.

Implicitamente viene dotata dei metodo `__iter__` e `__next__`

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```

```
for y in range(10):
```



```
    ...
```

vien chiamata la funzione

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

for y in range(10):

def range(stop):      ←      ...

    i = 0

    while i < stop:

        yield i

        i = i + 1

                                inizia l'esecuzione

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```

```
for y in range(10):
```

```
    ...
```



viene ritornato il primo valore e si interrompe l'esecuzione della funzione

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```

```
for y in range(10):
```

```
    ...
```



y assume tale valore

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```

```
for y in range(10):
```

```
    ...
```



viene utilizzato il valore e termina  
la prima iterazione del ciclo

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```

```
for y in range(10):
```



```
    ...
```

inizia la seconda iterazione

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```



```
for y in range(10):
```

```
    ...
```

viene ripresa l'esecuzione della  
funzione da dove si era interrotta

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```

```
for y in range(10):
```

```
    ...
```



viene ripresa l'esecuzione della  
funzione da dove si era interrotta

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```

```
for y in range(10):
```

```
    ...
```



viene ritornato il secondo valore

# Generatori - Funzioni generatrici

Esempio: range() fai da te!

```
def range(stop):
```

```
    i = 0
```

```
    while i < stop:
```

```
        yield i
```

```
        i = i + 1
```



```
for y in range(10):
```

```
    ...
```

Alla fine di tutto viene sollevata  
l'eccezione StopIteration per  
fermare il ciclo

# Generatori - Espressioni generatrici

I Generatori possono anche essere creati con una sintassi molto stringata.

Si parla di **espressioni generatrici**

Sintassi: (espressione **for** variabili **in** sequenza)

Esempio: (**x\*\*2 for x in range(10)**)

il generatore ritornerà ad ogni iterazione il quadrato dei numeri da 0 a 9

# List comprehension

Una sintassi simile (usando [] invece che ()) può essere usata per generare liste in modo immediato:

```
a = [x**2 for x in range(10)]
```

a conterrà una lista dei quadrati da 0 a 9

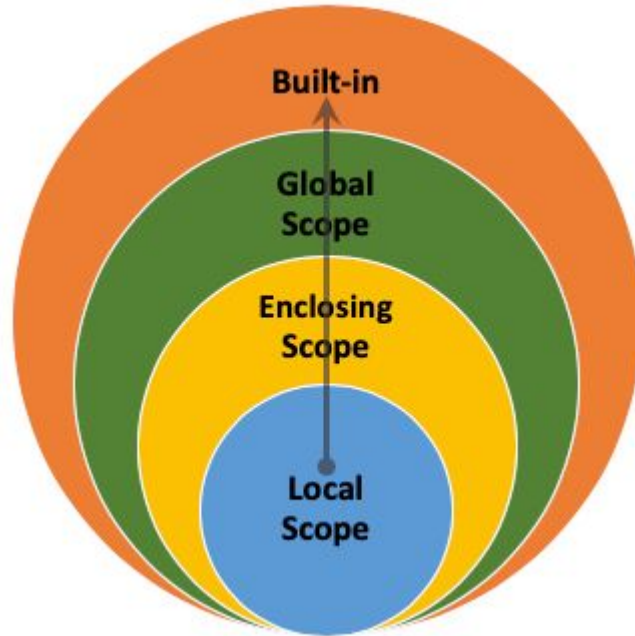
# Scope

Lo scope determina quali variabili (e entità in generali) sono visibili in un dato punto del codice.

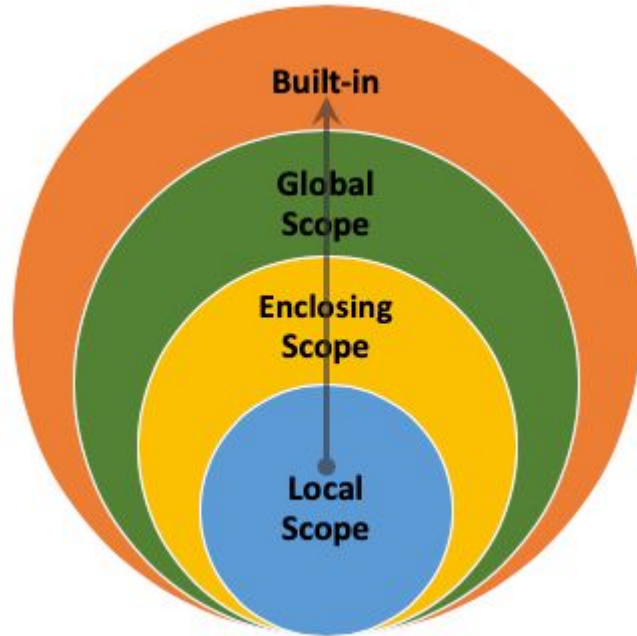
In Python gli scope sono 'concentrici' e la risoluzione è molto semplice.

Se un'entità non viene trovata nello scope più interno si passa a quello immediatamente più esterno e così via

# Scope



# Scope



- Built-in: entità sempre disponibili. Es. print, disponibile non appena apriamo la shell
- Global: entità definite nel codice (non dentro a funzioni o blocchi)
- Enclosing: entità incapsulate per effetto della closure
- Local: entità definite dentro il blocco di codice in cui si è

# Scope

```
print('ciao')
```

- **Built-in: entità sempre disponibili. Es. print, disponibile non appena apriamo la shell**
- Global: entità definite nel codice (non dentro a funzioni o blocchi)
- Enclosing: entità incapsulate per effetto della closure
- Local: entità definite dentro il blocco di codice in cui si è

# Scope

```
print('ciao')
```

```
a = 1
```

- Built-in: entità sempre disponibili. Es. print, disponibile non appena apriamo la shell
- **Global: entità definite nel codice (non dentro a funzioni o blocchi)**
- Enclosing: entità incapsulate per effetto della closure
- Local: entità definite dentro il blocco di codice in cui si è

# Scope

```
print('ciao')
```

```
a = 1
```

```
def encaps():
```

```
    b = 2
```

```
    def local():
```

```
        c = 3 + b
```

- Built-in: entità sempre disponibili. Es. print, disponibile non appena apriamo la shell
- Global: entità definite nel codice (non dentro a funzioni o blocchi)
- **Enclosing: entità incapsulate per effetto della closure**
- Local: entità definite dentro il blocco di codice in cui si è

# Scope

```
print('ciao')
```

```
a = 1
```

```
def encaps():
```

```
    b = 2
```

```
    def local():
```

```
        c = 3 + b
```

- Built-in: entità sempre disponibili. Es. print, disponibile non appena apriamo la shell
- Global: entità definite nel codice (non dentro a funzioni o blocchi)
- Enclosing: entità incapsulate per effetto della closure
- **Local: entità definite dentro il blocco di codice in cui si è**

# Scope

```
print('ciao')
```

```
a = 1
```

```
def encaps():
```

```
    b = 2
```

```
    def local():
```

```
        c = 3 + b
```

```
def another():
```

```
    d = 0
```

In questo punto del codice posso vedere  
print, a, b, c

- print -> scope built-in
- a -> scope globale
- b -> scope enclosed
- c -> scope locale

Non vedo d in quanto, in questo punto non  
è in nessuno scope

# Scope

```
print('ciao')
```

```
a = 1
```

```
def encaps():
```

```
    b = 2
```

```
    def local():
```

```
        c = 3 + b
```

```
def another():
```

```
    d = 0
```

Meccanismo di risoluzione:

- si parte dal Local
- poi si guarda enclosed
- poi global
- infine built-in

# Scope

```
print('ciao')
```

```
x = 1
```

```
def encaps():
```

```
    x = 2
```

```
    def local():
```

```
        x = 3
```

```
def another():
```

```
    x = 0
```

Si possono chiamare le variabili con lo stesso nome 'nascondendo' quelle degli scope più esterni

- si guarda lo scope più interno
- solo se quella entità non vi è si passa ai successivi

# Scope

```
print('ciao')
```

```
x = 1
```

```
def encaps():
```

```
    x = 2
```

```
    def local():
```

```
        global x
```

```
        x = 3
```

```
def another():
```

```
    x = 0
```

Si può forzare l'utilizzo di un'entità dello scope globale con la parola chiave **global**

- **in questo caso x si riferisce al 'primo' x**

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x = 'ciao'  
    print(x)
```

```
x = 5  
fun(x)  
print(x)
```

Anche gli argomenti delle funzioni entrano nel Local scope (altrimenti non li potremmo usare)

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x = 'ciao'  
    print(x)
```

```
x = 5  
fun(x)  
print(x)
```

Anche gli argomenti delle funzioni entrano nel Local scope (altrimenti non li potremmo usare)

output:

5

ciao

5

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x = 'ciao'  
    print(x)
```

```
x = 5  
fun(x)  
print(x)
```

output:

5

ciao

5

All'interno della funzione viene cambiata la  
x locale

Al di fuori rimane la x di prima

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x = 'ciao'  
    print(x)
```

```
x = 5  
fun(x)  
print(x)
```

E come se la x sia copiata nel parametro della funzione.

Passaggio per valore

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x.append(3)  
    print(x)
```

```
x = [1,2]  
fun(x)  
print(x)
```

Output

?

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x.append(3)  
    print(x)
```

```
x = [1,2]  
fun(x)  
print(x)
```

Output

[1,2]

[1,2,3]

**[1,2,3]**

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x.append(3)  
    print(x)
```

```
x = [1,2]  
fun(x)  
print(x)
```

Output

[1,2]

[1,2,3]

**[1,2,3]**

Come se fosse passato per riferimento

# Scope - argomenti funzioni

Abbiamo detto che in Python è tutto un oggetto.

Come mai abbiamo due comportamenti differenti?

# Scope - argomenti funzioni

In realtà in Python viene **copiato il riferimento all'oggetto**

Quindi nella x della funzione vi è il riferimento allo stesso oggetto puntato dalla x globale

# Scope - argomenti funzioni

In realtà in Python viene **copiato il riferimento all'oggetto**

Quindi nella x della funzione vi è il riferimento allo stesso oggetto puntato dalla x globale

Se si chiama **append** lo si chiama sullo stesso oggetto poiché tutte e due le x puntano al medesimo oggetto

```
x.append(3)
```

# Scope - argomenti funzioni

In realtà in Python viene **copiato il riferimento all'oggetto**

Quindi nella x della funzione vi è il riferimento allo stesso oggetto puntato dalla x globale

Quando si ri-assegna x si cambia il riferimento della x locale che ora punterà al nuovo oggetto

x = 3

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x = 'ciao'  
    print(x)
```

```
x = 5  
fun(x)  
print(x)
```



| <b>Variabile</b> | <b>Punta a</b> |
|------------------|----------------|
| x globale        | 5              |
| x locale         | non esiste     |

# Scope - argomenti funzioni

```
def fun(x):  
    print(x) ←  
    x = 'ciao'  
    print(x)
```

```
x = 5  
fun(x)  
print(x)
```

| Variabile | Punta a                            |
|-----------|------------------------------------|
| x globale | 5                                  |
| x locale  | 5 (è stato copiato il riferimento) |

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x = 'ciao' ←  
    print(x)
```

```
x = 5  
fun(x)  
print(x)
```

| Variabile | Punta a                                  |
|-----------|------------------------------------------|
| x globale | 5                                        |
| x locale  | 'ciao' (è stato cambiato il riferimento) |

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x = 'ciao'  
    print(x)
```



```
x = 5  
fun(x)  
print(x)
```

| Variabile | Punta a |
|-----------|---------|
| x globale | 5       |
| x locale  | 'ciao'  |

# Scope - argomenti funzioni

```
def fun(x):  
    print(x)  
    x = 'ciao'  
    print(x)
```

```
x = 5  
fun(x)  
print(x)
```



| <b>Variabile</b> | <b>Punta a</b> |
|------------------|----------------|
| x globale        | 5              |
| x locale         | non esiste più |

# Gestione delle eccezioni

Come in altri linguaggi, Python gestisce gli errori attraverso il concetto di Eccezione.

Nell'ambiente di runtime (RE), viene definita una classe madre che rappresenta eccezioni software generiche.

Ogni anomalia a runtime è associata a una sottoclasse specifica della classe madre.

Le eccezioni possono essere sollevate in due modi:

1. **Automaticamente:** Queste eccezioni si verificano in seguito a anomalie a runtime causate da istruzioni (ad esempio, divisione per zero).
2. **Manualmente:** I programmatori possono sollevare manualmente eccezioni utilizzando istruzioni come "throw" o "raise".

In caso di eccezione, viene eseguito del codice di gestione. Se non è presente un gestore specifico, verrà utilizzato un gestore predefinito che stamperà lo stack delle chiamate ed uscirà dal programma.

# Gestione delle eccezioni

In Python la sintassi per catturare un'eccezione è la seguente:

**try:**

**codice**

**except Classe\_eccezione as identificativo:**

**codice**

**except Altraclasse\_eccezione as identificativo:**

**codice**

**else:**

**codice**

**finally:**

**codice**

# Gestione delle eccezioni

In Python la sintassi per catturare un'eccezione è la seguente:

```
try:
    codice
except Classe_eccezione as identificativo:
    codice
except Altraclasse_eccezione as identificativo:
    codice
else:
    codice
finally:
    codice
```

← viene eseguito

# Gestione delle eccezioni

In Python la sintassi per catturare un'eccezione è la seguente:

**try:**

**codice**

**except Classe\_eccezione as identificativo:**

**codice**



**except Altraclasse\_eccezione as identificativo:**

**codice**

**else:**

**codice**

**finally:**

**codice**

viene eseguito se nel blocco try  
viene sollevata un'eccezione  
del tipo Classe\_eccezione

# Gestione delle eccezioni

In Python la sintassi per catturare un'eccezione è la seguente:

**try:**

**codice**

**except Classe\_eccezione as identificativo:**

**codice**

**except Altraclasses\_eccezione as identificativo:**

**codice**



**else:**

**codice**

**finally:**

**codice**

viene eseguito se nel blocco try  
viene sollevata un'eccezione  
del tipo Altraclasses\_eccezione

# Gestione delle eccezioni

In Python la sintassi per catturare un'eccezione è la seguente:

**try:**

codice

**except Classe\_eccezione as identificativo:**

codice

**except Altraclasse\_eccezione as identificativo:**

codice

**except:**

codice

**else:**

codice

**finally:**

codice

viene eseguito se nel blocco try  
viene sollevata un'eccezione  
non catturata esplicitamente  
prima



# Gestione delle eccezioni

In Python la sintassi per catturare un'eccezione è la seguente:

**try:**

codice

**except Classe\_eccezione as identificativo:**

codice

**except Altraclasse\_eccezione as identificativo:**

codice

**except:**

codice

**else:**

codice

**finally:**

codice



viene eseguito in ogni caso

# Gestione delle eccezioni

In Python la sintassi per catturare un'eccezione è la seguente:

**try:**

codice

**except** `Classe_eccezione as identificativo`:

codice

**except** `Altraclasse_eccezione as identificativo`:

codice

**except:**

codice

**else:**

codice

**finally:**

codice

possono esserci più coppie  
classe identificativo se si vuole  
utilizzare lo stesso codice per  
gestirle

# Gestione delle eccezioni

All'interno di una funzione (o blocco) si sollevano le eccezioni con il comando **raise**

```
raise NameException(arg1,arg2)
```

# Decoratori

Si usano i decoratori per aggiungere funzionalità ad una funzione.

Es.

misurare il tempo di esecuzione di una funzione.

```
def func(x):
```

```
    time.sleep( random.random()*x )
```

```
start = time.time()
```

```
func(5)
```

```
end = time.time()
```

```
print ("ci ha messo ", end - start, " secondi")
```

# Decoratori

Se si voglio cronometrare tante funzioni diventa noioso e poco leggibile.

Possiamo definire una funzione per misure il tempo

```
def time_function(function):
```

```
    def new_function():
```

```
        start = time.time()
```

```
        value = function()
```

```
        end = time.time()
```

```
        print ("ci ha messo ", end - start, " secondi")
```

```
        return value
```

```
    return new_function
```

```
f = time_function( func )  
f()
```

# Decoratori

La funzione però potrebbe necessitare di un parametro

```
def time_function(function):
```

```
    def new_function(x):
```

```
        start = time.time()
```

```
        value = function(x)
```

```
        end = time.time()
```

```
        print ("ci ha messo ", end - start, " secondi")
```

```
        return value
```

```
    return new_function
```

```
f = time_function( func )  
f(5)
```

# Decoratori

La funzione però potrebbe necessitare di un parametro... o più parametri

```
def time_function(function):
```

```
    def new_function(args*, kwargs**):
```

```
        start = time.time()
```

```
        value = function(args*, kwargs**)
```

```
        end = time.time()
```

```
        print ("ci ha messo ", end - start, " secondi")
```

```
        return value
```

```
return new_function
```

```
f = time_function( func )  
f(5,2)
```

# Decoratori

Con questa funzione copriamo tutti i casi... I decorator semplificano il suo utilizzo

Si definisce una funzione che prenda come parametro una funzione e restituisca una funzione

```
def time_function(function):  
    def new_function(args*, kwargs**):  
        start = time.time()  
        value = function(args*, kwargs**)  
        end = time.time()  
        print ("ci ha messo ", end - start, " secondi")  
        return value  
    return new_function
```

# Decoratori

Con questa funzione copriamo tutti i casi... I decorator semplificano il suo utilizzo

La si applica ad altre funzioni con la sintassi @

**@time\_function**

**def func4(x,y,z):**

**time.sleep( random.random()\*x+(y-z) )**

# Decoratori

A questo punto func4 non sarà la funzione definita dall'utente ma la funzione restituita da time\_function

Sarà una funzione arricchita

**@time\_function**

**def func4(x,y,z):**

**time.sleep( random.random()\*x+(y-z) )**